Walking Through the Branches -- And Making the Walk Smarter

Lecture 10

Data Structures and Algorithms

Part 1: What is Tree Traversal?

The Problem: Visiting Every Node

A tree can hold hundreds or millions of nodes. To do anything useful -- printing, searching, copying, evaluating -- we need a systematic way to **visit every node exactly once**.

This is called **traversal**.

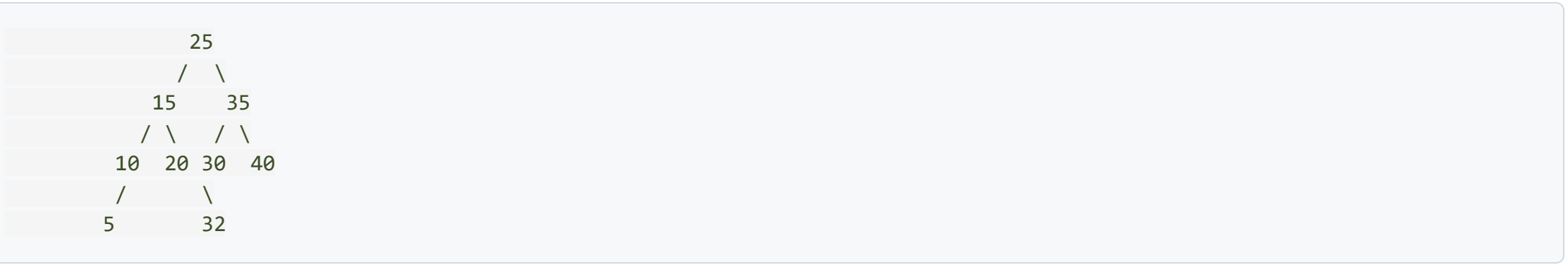
But unlike a linked list, a tree branches. There is no single "next" node. At every internal node we face a choice: go left, go right, or process the current node.

The order in which we make that choice gives us different traversals:

Traversal	Strategy	Data Structure Used
In-order	Left, Node, Right	Recursion (stack)
Pre-order	Node, Left, Right	Recursion (stack)
Post-order	Left, Right, Node	Recursion (stack)
Level-order	Level by level	Queue

The Reference Tree

We will use the same tree for **all traversal examples** in this lecture. Study it carefully:



9 nodes, height 3. Keep this tree in front of you as we trace each traversal.

Some useful facts about this tree:

- It has **4 leaves:** 5, 20, 32, 40
- It has **5 internal nodes:** 25, 15, 35, 10, 30
- It has **9 x 2 = 18 pointers**, but only **8 edges** -- so **10 NULL pointers** are wasted

Part 2: Depth-First Traversals

In-Order Traversal (LNR)

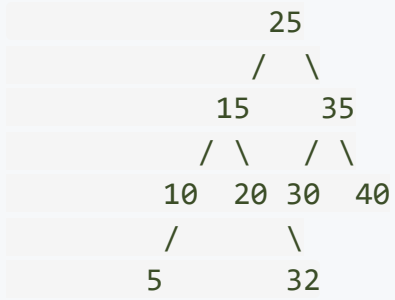
Rule: For every node, visit the **left subtree** first, then the **node itself**, then the **right subtree**.

```
void inorder(struct Node* node) {
    if (node == NULL) return;
    inorder(node->left);        // L - go left
    printf("%d ", node->data);  // N - visit node
    inorder(node->right);       // R - go right
}
```

Why "In-order"? The node is visited **in between** its subtrees.

The most important property: If the tree is a Binary Search Tree (BST), in-order traversal visits nodes in **sorted ascending order**. Our reference tree is a BST, so the output will be sorted!

In-Order: Step-by-Step Trace




```
inorder(25)
  inorder(15)
    inorder(10)
      inorder(5)
        inorder(NULL) -> return
        PRINT 5
        inorder(NULL) -> return
      PRINT 10
      inorder(NULL) -> return
    PRINT 15
    inorder(20)
      inorder(NULL) -> return
      PRINT 20
      inorder(NULL) -> return
  PRINT 25
  inorder(35)
    inorder(30)
      inorder(NULL) -> return
      PRINT 30
      inorder(32)
        inorder(NULL) -> return
        PRINT 32
        inorder(NULL) -> return
```

Pre-Order Traversal (NLR)

Rule: Visit the **node first**, then the left subtree, then the right subtree.

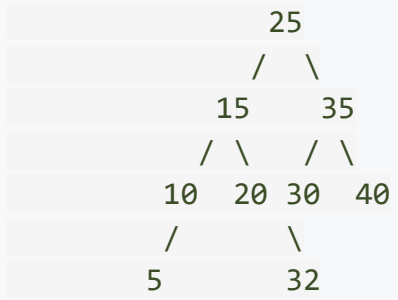
```
void preorder(struct Node* node) {
    if (node == NULL) return;
    printf("%d ", node->data);  // N - visit node FIRST
    preorder(node->left);       // L - then go left
    preorder(node->right);      // R - then go right
}
```

Why "Pre-order"? The node is visited **before** (pre) its subtrees.

Use Cases

- **Copying a tree:** If you process the root before its children, you can create the copy top-down -- create the parent node first, then attach children.
- **Serialization:** Pre-order output can be used to reconstruct the tree later.
- **Expression trees:** Pre-order gives **prefix notation** (Polish notation).

Pre-Order: Step-by-Step Trace



The root is always printed **first** in pre-order. Then we dive left as deep as possible, backtrack, and go right.

```
Visit 25, go left:
  Visit 15, go left:
    Visit 10, go left:
      Visit 5, no children -> backtrack
    go right: NULL -> backtrack
  go right:
    Visit 20, no children -> backtrack
go right:
  Visit 35, go left:
    Visit 30, go left: NULL
    go right:
      Visit 32, no children -> backtrack
  go right:
    Visit 40, no children -> backtrack
```

Output: 25 15 10 5 20 35 30 32 40

Notice: the root (25) comes first, then the entire left subtree, then the entire right subtree.

Post-Order Traversal (LRN)

Rule: Visit the left subtree, then the right subtree, then the **node last**.

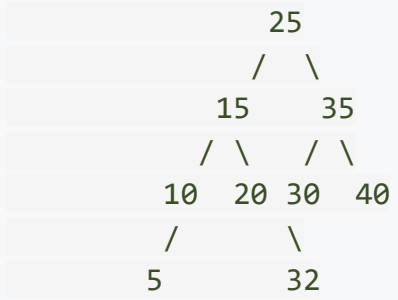
```c
void postorder(struct Node* node) {
    if (node == NULL) return;
    postorder(node->left);      // L - go left first
    postorder(node->right);     // R - then go right
    printf("%d ", node->data);  // N - visit node LAST
}
```

Why "Post-order"? The node is visited **after** (post) its subtrees.

Use Cases

- **Deleting a tree:** You must delete children **before** their parent -- otherwise you lose the pointers to the children and leak memory.
- **Calculating folder sizes:** Compute child folder sizes first, then sum them for the parent.
- **Expression trees:** Post-order gives **postfix notation** (Reverse Polish notation) -- the same notation we evaluated using stacks in the previous unit!

Post-Order: Step-by-Step Trace



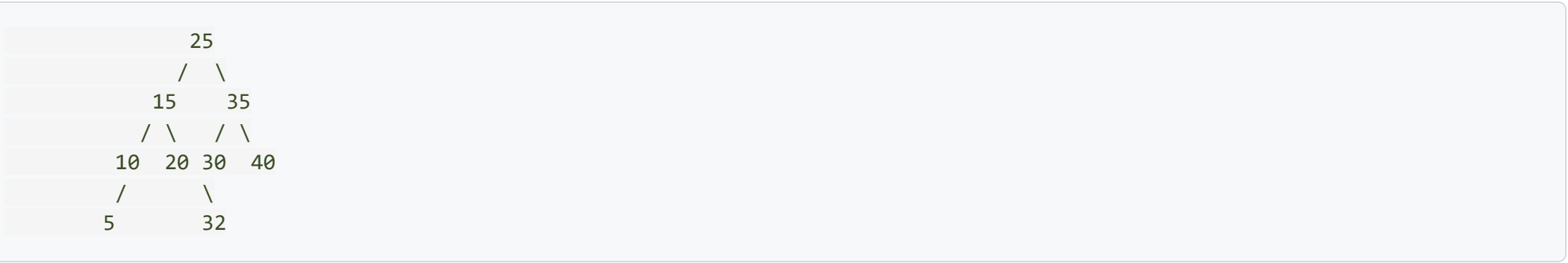

```
Dive left all the way:
  Dive into 25 -> 15 -> 10 -> 5
    5 has no children -> PRINT 5
  Back to 10, right is NULL -> PRINT 10
  Back to 15, go right to 20
    20 has no children -> PRINT 20
  PRINT 15
Back to 25, go right to 35
  Go left to 30, left is NULL
    Go right to 32
      32 has no children -> PRINT 32
    PRINT 30
  Go right to 40
    40 has no children -> PRINT 40
  PRINT 35
PRINT 25
```

Output: 5 10 20 15 32 30 40 35 25

Notice: the root (25) comes **last**. Leaves are always visited before their parents.

The Three DFS Traversals -- Comparison

All three visit the **same nodes** -- the only difference is **when** we process each node:



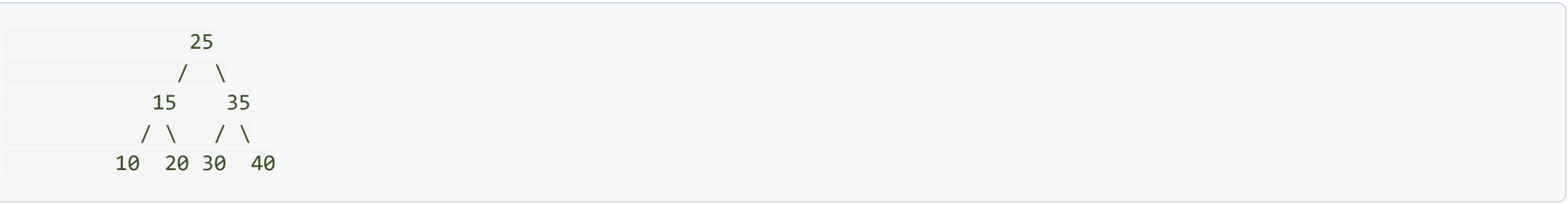
Traversal	Order	Output
In-order (LNR)	Left, **Node**, Right	5 10 15 20 **25** 30 32 35 40
Pre-order (NLR)	**Node**, Left, Right	**25** 15 10 5 20 35 30 32 40
Post-order (LRN)	Left, Right, **Node**	5 10 20 15 32 30 40 35 **25**

Complexity for all three:

- **Time:** O(n) -- every node visited exactly once
- **Space:** O(h) -- recursion depth equals the height of the tree
  - Best case (balanced): O(log n)
  - Worst case (skewed): O(n)

A Trick to Read Traversals from a Diagram

Imagine tracing a path that hugs the **outside boundary** of the tree. As you walk around the tree, you pass each node **three times**:



- **Pre-order:** Print a node the **1st time** you pass it (on the left side)
- **In-order:** Print a node the **2nd time** you pass it (from below)
- **Post-order:** Print a node the **3rd time** you pass it (on the right side)

This visual method works for any binary tree and can be faster than tracing recursion for exam questions.

Part 3: Level-Order Traversal

Level-Order: Breadth-First Search on a Tree

Rule: Visit all nodes at depth 0, then depth 1, then depth 2, and so on. Within each level, go **left to right**.

```
              25          <- Level 0: visit 25
             /  \
           15    35       <- Level 1: visit 15, 35
          / \   / \
        10  20 30  40     <- Level 2: visit 10, 20, 30, 40
        /      \
       5       32         <- Level 3: visit 5, 32
```

Output: 25 15 35 10 20 30 40 5 32

This is **not** a depth-first traversal. It does not use recursion. Instead, it uses a **Queue** -- exactly the BFS pattern we studied in the Queues unit.

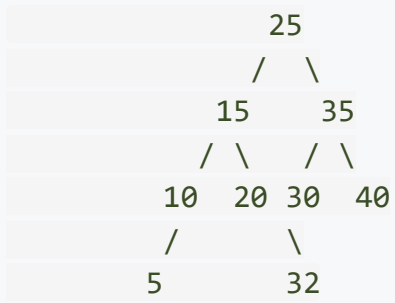
Level-Order Algorithm

```c
void levelorder(struct Node* root) {
    if (root == NULL) return;

    // Use a queue to track nodes to visit
    struct Node* queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;    // Start with the root

    while (front < rear) {
        struct Node* current = queue[front++];    // Dequeue
        printf("%d ", current->data);             // Visit

        if (current->left  != NULL) queue[rear++] = current->left;   // Enqueue left
        if (current->right != NULL) queue[rear++] = current->right;  // Enqueue right
    }
}
```

Why a queue? The queue processes nodes in FIFO order. By enqueuing all children of the current level before moving on, nodes are automatically visited level by level.

Complexity: O(n) time, O(w) space -- where w is the maximum width of the tree.

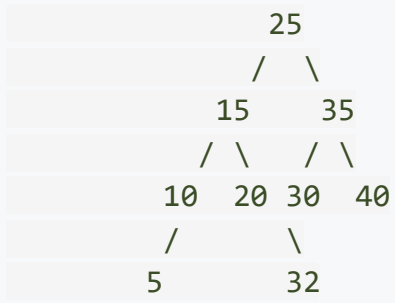
Level-Order: Queue Trace



Step	Dequeue	Print	Enqueue Children	Queue After
1	25	25	15, 35	[15, 35]
2	15	15	10, 20	[35, 10, 20]
3	35	35	30, 40	[10, 20, 30, 40]
4	10	10	5	[20, 30, 40, 5]
5	20	20	(none)	[30, 40, 5]
6	30	30	32	[40, 5, 32]
7	40	40	(none)	[5, 32]
8	5	5	(none)	[32]
9	32	32	(none)	[] -- done

Output: 25 15 35 10 20 30 40 5 32

All Four Traversals -- Summary



Traversal	Type	Output	Key Use
In-order	DFS	5 10 15 20 25 30 32 35 40	Sorted output from BST
Pre-order	DFS	25 15 10 5 20 35 30 32 40	Copy tree, serialize
Post-order	DFS	5 10 20 15 32 30 40 35 25	Delete tree, evaluation
Level-order	BFS	25 15 35 10 20 30 40 5 32	Shortest path, levels

The fundamental distinction:

- **DFS** (in/pre/post) uses a **Stack** (via recursion) -- explores depth before breadth
- **BFS** (level-order) uses a **Queue** -- explores breadth before depth

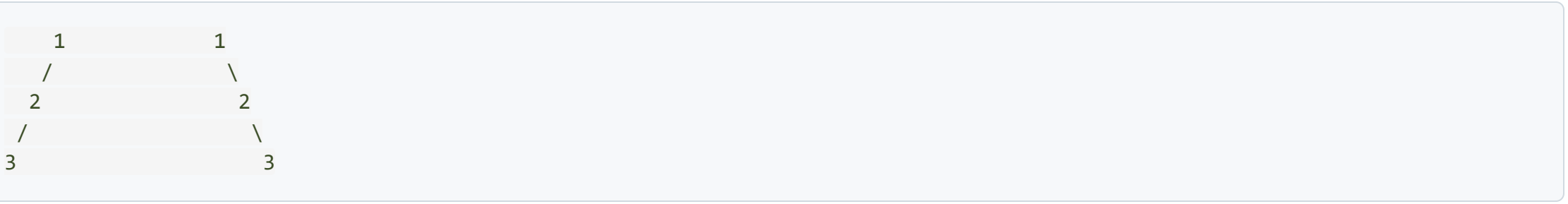
Part 4: Reconstructing Trees from Traversals

Can We Rebuild a Tree from Its Traversal?

Given just one traversal sequence, can we reconstruct the original tree?

No. Multiple different trees can produce the same traversal output.

Example -- both of these trees give **pre-order = 1, 2, 3:**



We need TWO traversal sequences to uniquely determine a binary tree, and one of them must be **in-order**.

Reconstruction: In-Order + Pre-Order

Given:

- In-order: 5 10 15 20 25 30 32 35 40
- Pre-order: 25 15 10 5 20 35 30 32 40

The Algorithm:

1. The **first element** of pre-order is always the **root** --> root = **25**
2. Find **25** in the in-order sequence -- everything to its **left** is the left subtree, everything to its **right** is the right subtree
3. Recurse on each half

```
In-order:   [5  10  15  20]  25  [30  32  35  40]
                 LEFT              RIGHT

Pre-order:  25  [15  10  5  20]  [35  30  32  40]
                  LEFT              RIGHT
```

Root = 25, Left subtree has nodes {5, 10, 15, 20}, Right subtree has nodes {30, 32, 35, 40}.

Reconstruction: Worked Example (continued)

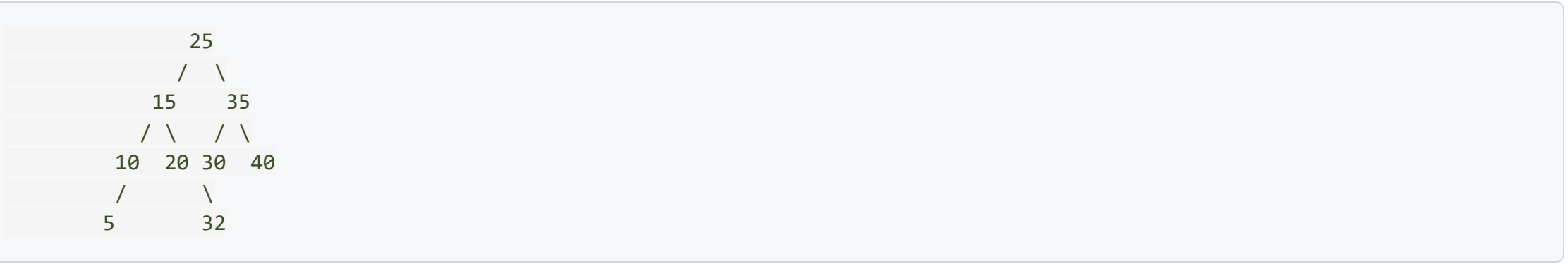
Left subtree: In-order = [5, 10, 15, 20], Pre-order = [15, 10, 5, 20]

Root = **15** (first in pre-order). Find 15 in in-order:

```
In-order:  [5  10]  15  [20]       Pre-order:  15  [10  5]  [20]
```

Root = 15, Left = {5, 10}, Right = {20}

Continue recursing until single nodes remain:



We recover the original tree!

Which Pairs Work?

Pair	Can Reconstruct?	Why?
In-order + Pre-order	Yes	Pre-order gives root; in-order splits left/right
In-order + Post-order	Yes	Post-order gives root (last element); in-order splits
Pre-order + Post-order	No	Cannot distinguish left from right subtree
In-order + Level-order	Yes	Level-order gives root; in-order splits

In-order is essential because it is the only traversal that tells us which nodes belong to the left subtree and which belong to the right subtree of any given root.

Exception: If the tree is a **full binary tree** (every node has 0 or 2 children), then pre-order + post-order is sufficient.

Part 5: Threaded Binary Trees

The Wasted Pointer Problem

Recall from last lecture: a binary tree with n nodes has $2n$ pointers but only $n - 1$ edges.

That leaves $n + 1$ **NULL pointers** -- more than half!

```
              25
             /  \
           15    35
          / \   / \
        10  20 30  40              9 nodes, 18 pointers
        /      \                   8 edges, 10 NULLs!
       5       32

  Nodes with NULL pointers: 5 (both), 20 (both), 32 (both),
                            10 (right only), 30 (left only), 40 (both)
```

Question: Can we use these NULL pointers for something useful?

Answer: Yes! We can turn them into **threads** -- pointers to the in-order predecessor or successor. This allows us to traverse the tree **without recursion and without a stack**.

What is a Threaded Binary Tree?

Instead of leaving NULL pointers empty, we replace them with **threads** -- shortcuts to the in-order predecessor or successor.

Single-threaded (right-threaded): NULL right pointers point to the in-order **successor**.

Double-threaded: NULL left pointers point to the in-order **predecessor**, and NULL right pointers point to the in-order **successor**.

But how do we tell threads apart from real child pointers?

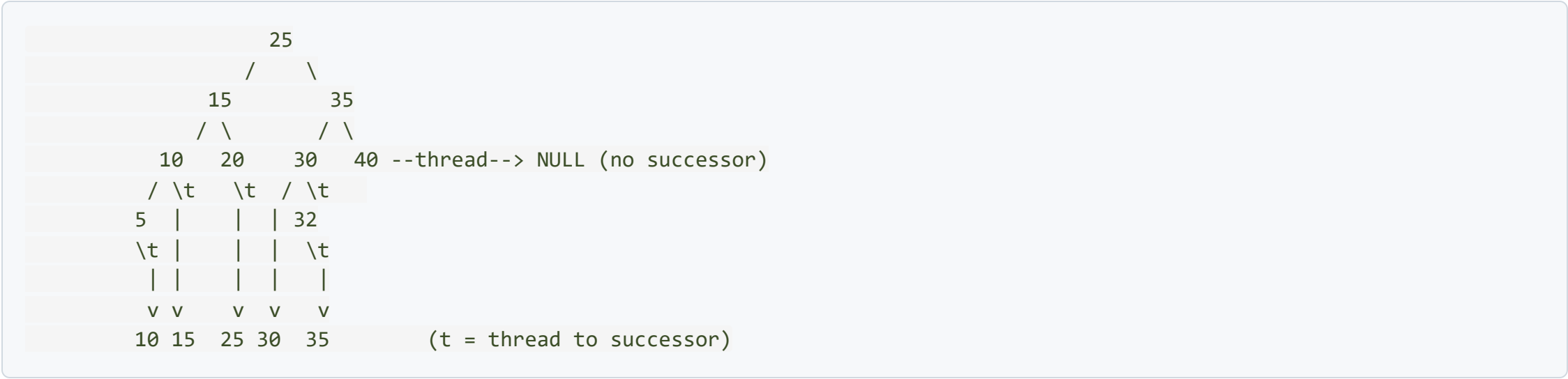
We add a **flag** to each pointer:

```
struct ThreadedNode {
    int data;
    struct ThreadedNode* left;
    struct ThreadedNode* right;
    int leftThread;    // 1 = thread (predecessor),  0 = real child
    int rightThread;   // 1 = thread (successor),    0 = real child
};
```

Right-Threaded Binary Tree: Example

In-order sequence: 5, 10, 15, 20, 25, 30, 32, 35, 40

Replace each NULL right pointer with a thread to the in-order **successor**:



- Node 5: right is NULL --> thread to **10** (successor)
- Node 10: right is NULL --> thread to **15** (successor)
- Node 20: right is NULL --> thread to **25** (successor)
- Node 30: left is NULL --> left stays NULL (no predecessor for now, or thread in double-threaded)
- Node 32: right is NULL --> thread to **35** (successor)
- Node 40: right is NULL --> thread to NULL (40 is the last node)

In-Order Traversal Using Threads

With a right-threaded tree, we can perform in-order traversal with **O(1) extra space** -- no recursion, no stack!

Algorithm:

1. Start at the **leftmost node** of the tree
2. **Visit** the current node
3. If the right pointer is a **thread** --> follow it to the successor
4. If the right pointer is a **real child** --> go to the leftmost node in that right subtree
5. Stop when there is no successor (thread points to NULL)

```
Start at leftmost: 5
Visit 5  -> right is thread  -> follow to 10
Visit 10 -> right is thread  -> follow to 15
Visit 15 -> right is child   -> go to leftmost of right subtree -> 20
Visit 20 -> right is thread  -> follow to 25
Visit 25 -> right is child   -> go to leftmost of right subtree -> 30
Visit 30 -> right is child   -> go to leftmost of right subtree -> 32
Visit 32 -> right is thread  -> follow to 35
Visit 35 -> right is child   -> go to leftmost of right subtree -> 40
Visit 40 -> right is thread  -> NULL -> STOP
```

Output: 5 10 15 20 25 30 32 35 40 -- correct in-order, with O(1) space!

Threaded Traversal: C Code

```c
// Find the leftmost node in a subtree
struct ThreadedNode* leftmost(struct ThreadedNode* node) {
    if (node == NULL) return NULL;
    while (node->left != NULL && node->leftThread == 0) {
        node = node->left;
    }
    return node;
}

// In-order traversal using threads -- no recursion, no stack!
void inorderThreaded(struct ThreadedNode* root) {
    struct ThreadedNode* current = leftmost(root);

    while (current != NULL) {
        printf("%d ", current->data);

        if (current->rightThread == 1) {
            // Right pointer is a thread -> follow it to successor
            current = current->right;
        } else {
            // Right pointer is a real child -> go to leftmost of right subtree
            current = leftmost(current->right);
        }
    }
}
```

Standard vs Threaded: Side-by-Side

Feature	Standard Traversal	Threaded Traversal
Extra space	O(h) -- recursion stack	O(1) -- no stack needed
Time	O(n)	O(n)
Finding successor	Walk up the call stack	Follow the thread -- O(1)
Node structure	`data`, `left`, `right`	`data`, `left`, `right` + 2 flags
Insertion complexity	Simple	Must update threads
Deletion complexity	Simple	Must update threads

When are threads worth it?

- When you need **frequent successor/predecessor** queries (e.g., iterating over a sorted set)
- When you need **O(1) space** traversal (embedded systems, tight memory)
- When insertions and deletions are **rare** compared to traversals

Morris Traversal -- A Brief Mention

There is a clever algorithm called **Morris Traversal** that achieves O(1) space in-order traversal on a **standard** binary tree (no thread flags needed).

The idea:

1. Before going left, create a **temporary thread** from the rightmost node in the left subtree back to the current node
2. When you encounter this thread again, you know you have finished the left subtree -- remove the thread and move right

```
Temporarily thread:  rightmost of left subtree --> current node
After processing:    remove the thread, restore the original tree
```

Result: O(n) time, O(1) space, and the tree is **restored to its original state** after traversal.

This is an advanced technique -- for now, just know that it exists. It combines the space efficiency of threaded trees with the simplicity of not needing extra flags.

Key Takeaways

What We Covered Today

1. Four Traversals of a Binary Tree

- **In-order (LNR):** Left, Node, Right -- gives sorted output for BSTs
- **Pre-order (NLR):** Node, Left, Right -- useful for copying and serializing trees
- **Post-order (LRN):** Left, Right, Node -- useful for deletion and evaluation
- **Level-order (BFS):** Queue-based, level by level -- useful for shortest paths

2. Tree Reconstruction

- One traversal is not enough -- need two, and one must be in-order
- In-order + pre-order (or post-order or level-order) can reconstruct the tree

3. Threaded Binary Trees

- Use NULL pointers as threads to in-order successors/predecessors
- Enable O(1) space traversal without recursion or a stack
- Trade-off: simpler traversal, more complex insertion/deletion

Review Questions

1. Perform **in-order**, **pre-order**, and **post-order** traversals on this tree:

```
        50
       / \
     30   70
    / \     \
  20  40    80
```

2. Given **in-order = [D, B, E, A, F, C]** and **pre-order = [A, B, D, E, C, F]**, reconstruct the original binary tree.
3. Why does **pre-order + post-order** fail to uniquely reconstruct a binary tree? Give a counterexample.
4. A binary tree has 15 nodes. How many NULL pointers does it have? How many of these can be replaced with threads?
5. Explain why threaded binary trees can perform in-order traversal in O(1) space, while standard binary trees require O(h) space.
6. In a right-threaded binary tree, what does a node's right pointer point to if the `rightThread` flag is set to 1?

Next Lecture

Binary Search Trees -- Properties and Operations

- Adding the power of ordering to binary trees
- Search, insertion, and deletion in O(log n) -- when the tree cooperates
- The dangerous case of sorted input and the road to balanced trees